

FACULTY
OF
INFORMATION
AND
COMMUNICATION TECHNOLOGY

NATIONAL DIPLOMA: COMPUTER SYSTEMS ENGINEERING

PROJECT DESIGN III: PD2316D

DESIGN DOCUMENTATION

PROJECT TITLE:	**Biometric Student Attendance and Verification System**
LECTURER:	**NAEMuwanguzi Mark Ntume and Chunling Du**
STUDENT'S NAME:	**Lesiba Lucky**
STUDENT'S NUMBER:	**230325898**
EMAIL ADDRESS:	**lesibamanala793@gmail.com**
CONTACT NUMBER:	**0684846156**
SUBMISSION DATE:	**15-Sep-2026**

Introduction 3
Systems overview 3
Design considerations 4
Assumptions and dependencies 4
Related Software and Hardware 4
End-User Characteristics 5
General Constraints 5
Hardware and software environment 5
End-user environment 5
Resource availability 5
Requirements-driven constraints 6
Goals and guidelines 6
Fail-safe and Visible 6
Privacy by Design and Data Minimisation 6
Inclusivity 6
Speed Over Storage at the Terminal 7
KISS at the Terminal, Richness at the Server 7
Honest Measurement 7
Development Methods 7
Architectural Strategies 10
1. Distribution of Computation 10
2. Recogniser Selection 10
3. Matching Topology 10
4. Persistence and Distribution 11
5. Error Detection and Recovery 11
6. Interface Paradigm 11
7. Extensibility 11
SYSTEM ARCHITECTURE 13
Major System Responsibilities 13
Three-Tier Decomposition 13
Attendance Terminal 13
Backend 14
Dashboard 14
SUBSYSTEM ARCHITECTURE 18
Acquisition Task 18
HMI Task 18
Queue Manager 19
Network Client 19
RTC/Time Manager 19
State Machine 19
DETAILED SYSTEM DESIGN 20
Face Recognition Service 23
Attendance API 25
Functional Flow 27
Conclusion 38
Glossary 38
Bibliography 39

Introduction

This design document specifies the design of the biometric student attendance and verification system in sufficient detail to support implementation, technical review, and academic assessment. It documents the design decisions and structure, providing a consistent technical reference throughout the implementation process. The document therefore serves as the principal design reference for translating the project into a functional system.

The scope of this document covers the system and subsystem architecture, hardware and software design, data model, interfaces, and functional flow of the proposed system. It addresses the interaction between the biometric terminal, camera, and server components, together with the supporting data and software structures required for operation. The document does not cover the user manual or test results. These deliverables are treated as separate documents; therefore, out of scope.

The intended audience comprises the project supervisor, who will use the document as part of the project's technical assessment, and technically competent readers responsible for implementing, reviewing, or evaluating the system design. The document is also intended to provide a useful reference for future students who may extend or further develop the system. Its organization and level of detail are therefore intended to support both academic evaluation and practical understanding of the implemented design.

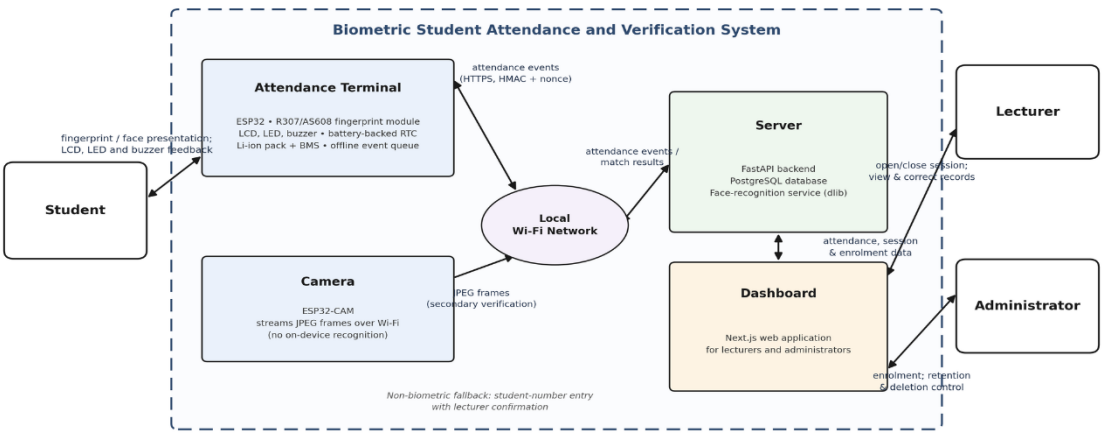
Systems overview

The Biometric Student Attendance and Verification system captures and verifies learner attendance during lectures and presents the resulting attendance information to authorized lecturers and administrators. The system is intended to provide a portable supervised means of recording attendance while reducing reliance on manual attendance capture.

The system is decomposed into three tiers: The terminal, camera, and server-side application. The terminal is the primary hardware that is responsible for fingerprint acquisition and on-module matching, local user feedback, timekeeping, event generation, and temporary offline storage. The camera tier uses an ESP32-CAM to stream JPEG frames over the local Wi-Fi network for use by the server-side face-recognition service; it performs no biometric recognition itself. The server tier provides the FastAPI backend, PostgreSQL database, Python-based face-recognition service, and Next.js dashboard, and is responsible for processing attendance events, performing confirmatory face recognition, storing records, and presenting attendance information. As illustrated in Figure 1, these three tiers operate together to complete the attendance capture and verification flow.

Identity is determined using 1 identification, with fingerprint matching as the primary biometric modality and face recognition as the confirmatory modality when the fingerprint result is insufficient or the finger cannot be acquired. The system is designed for operation within a single classroom per session using local Wi-Fi, with a portable battery-powered terminal capable of tolerating temporary network unavailability through offline event queuing and later synchronization.

Figure 1 – System Context Diagram



Design considerations

Several design issues had to be resolved before a complete design solution could be developed. The first concerns where biometric recognition should execute: on the microcontroller or on the server. This was resolved in favor of server-side processing because the ESP32 cannot host the required Python/OpenCV/dlib software stack, while recognition on the ESP32-CAM requires PSRAM and is significantly affected by varying illumination.

A second issue concerned whether the system should perform identification or verification, and how the biometric modalities should be combined. The design uses **1 identification**, with fingerprint as the primary modality and face recognition as the confirmatory modality, while limiting enrolment to approximately one class to reduce the false-accept probability associated with 1 matching.

The lawful processing of biometric information under POPIA was also required to be addressed before implementation. The design therefore requires appropriate consent, provides a mandatory non-biometric fallback, stores biometric templates rather than raw biometric images, and applies a defined retention and deletion rule.

The system also had to remain functional when network connectivity or mains power was unavailable. This was resolved through a local event queue with signed and idempotent deferred synchronisation, together with battery operation of the terminal to maintain attendance capture during temporary supply interruptions.

Finally, the design had to consider presentation attacks against both biometric modalities. The solution therefore includes bounded presentation attack detection testing aligned with **ISO/IEC 30107-3**, while explicitly treating the system as a supervised prototype and stating its security limitations rather than presenting it as access-control-grade authentication.

Assumptions and dependencies

Related Software and Hardware

The system depends on a combination of embedded hardware, development tools, and server-side software. The principal hardware and software dependencies are as follows:

- **ESP32 module and Arduino IDE toolchain:** The terminal software depends on the ESP32 development environment and compatible libraries for communication with the connected peripherals.
- **ESP32-CAM:** Provides JPEG image streaming over Wi-Fi and therefore depends on stable wireless connectivity and compatible camera firmware.
- **R307/AS608-class fingerprint module:** Provides fingerprint acquisition and on-module matching through UART. Its operation depends on the module's supported command set, enrolled template capacity, and serial interface.
- **16x2 I2C LCD and RTC module:** The LCD provides local feedback, while the RTC provides terminal-side time during offline operation. Both depend on reliable I2C communication and compatible libraries.
- **Li-ion battery pack with BMS and custom PCB:** The portable terminal depends on a protected rechargeable battery, charging circuitry, and the successful fabrication of the custom double-sided PCB.
- **Python, FastAPI, PostgreSQL, dlib/face recognition, and Next.js:** These components provide the server-side recognition, API, database, and dashboard functionality. Their operation depends on compatible software versions, required Python packages, and sufficient laptop resources.
- **KiCad and VS Code:** KiCad is used for PCB design and VS Code for software development. Development therefore depends on these tools and their associated project files, extensions, and libraries.

The server tier additionally depends on a local Wi-Fi network for communication between the terminal, camera, and server.

End-User Characteristics

The system is designed for three user roles with different levels of interaction:

- **Student:** An occasional user who is assumed to require no specialized training. Attendance capture should be completed within the five-second target, and the student may decline biometric enrolment and use the mandatory non-biometric fallback (POPIA).
- **Lecturer:** Responsible for opening attendance sessions, reviewing attendance information, and correcting records where necessary.
- **Administrator:** Responsible for enrolling students and managing biometric data retention and deletion in accordance with the defined requirements.

The functionality is expected to evolve beyond the initial prototype. Probable future changes include:

- Support for multiple attendance terminals.
- Integration with institutional systems such as student-information or learning-management systems.
- Extension of the system towards access-control applications.

These changes are treated as Future Work and do not affect the current implementation scope.

General Constraints

Several global constraints have a direct impact on the design and implementation:

Hardware and software environment

The limited processing capability of the ESP32 prevents it from hosting the Python/OpenCV/dlib face-recognition stack. This requires computationally intensive face processing to remain server-side, while the terminal performs fingerprint matching and local event handling.

The system also depends on the selected embedded libraries, server software stack, and a server-based environment. Consequently, software compatibility and resource requirements must be considered when selecting libraries, versions, and implementation methods.

End-user environment

The system is intended for a classroom environment where:

- **Lighting conditions may vary**, affecting face-recognition performance and therefore requiring fingerprint matching to remain the primary modality.
- **Students may form a queue**, requiring a fast verification process and the throughput target of at least 12 students per minute.
- **Campus Wi-Fi may become unavailable**, requiring offline event storage and later synchronization rather than making attendance dependent on continuous connectivity.
- **Load-shedding or mains-power loss may occur**, requiring the portable terminal to operate from its rechargeable battery.

Resource availability

The available resources impose practical limits on the design:

- The **R2 012.80 budget** limits component selection and favors low-cost, readily available hardware.
- Development by a **single developer** limits the amount of hardware and software complexity that can reasonably be implemented and tested.

- The **3–4 November 2026 final presentation deadline** constrains the development schedule and prioritizes the core vertical slice before optional functionality.
- The PCB is **home-etched and does not support fine-pitch fabrication**, limiting component selection and requiring layouts that can be manufactured using the available fabrication method. A breadboard implementation is therefore retained as a demonstration fallback.

Requirements-driven constraints

The design must satisfy the quantitative acceptance targets defined for the system, including biometric error rates, verification time, throughput, offline capacity and synchronisation, duplicate-record prevention and battery operating time. These targets directly influence component selection, algorithm configuration, communication protocols, database constraints and the design of the attendance workflow.

The system must also satisfy the applicable **POPIA and ethical requirements**. This constrains the collection, storage, retention, and deletion of biometric information and requires a non-biometric attendance fallback, appropriate consent, secure handling of templates, and controlled administrative operations.

Goals and guidelines

The design is guided by a set of principles that prioritize reliability, privacy, inclusivity, speed, and maintainability. These principles influence how the software behaves during normal operation, how it responds to failures, and how system resources are allocated between the terminal and server.

Fail Safe and Visible

The system shall prioritise preservation of attendance records over response speed.

- **Design rule:** The terminal shall indicate successful attendance only after server acknowledgement or after the event has been durably queued locally.

This is desirable because a delayed attendance record is preferable to a lost record. The approach also ensures that temporary network failures do not result in students believing that their attendance has been captured when no recoverable event exists.

Privacy by Design and Data Minimisation

The system shall collect and retain only the biometric information required for identification and verification.

- **Design rule:** Biometric templates rather than raw images shall be stored, face embeddings shall use pseudonymous identifiers and encryption at rest, and templates shall be deleted when the student completes or withdraws from the module.

This is desirable because biometric information cannot easily be replaced if compromised. Limiting the data collected and defining its retention period reduces privacy and security risks while supporting the POPIA requirements for appropriate processing and retention.

Inclusivity

The system shall provide a reliable attendance path regardless of whether biometric identification is available for a particular student.

- **Design rule:** A student shall not be denied attendance because they decline biometric enrolment or because their biometric cannot be acquired; the student-number fallback with lecturer confirmation shall therefore remain available.

This is desirable because biometric consent must be voluntary and because legitimate attendance should not depend entirely on successful biometric capture. The principle also improves accessibility for users for whom biometric acquisition may be temporarily unsuccessful.

Speed Over Storage at the Terminal

The terminal shall prioritise rapid attendance processing rather than extensive local data storage.

- **Design rule:** Fingerprint matching shall be performed on the fingerprint module, with the terminal using local storage primarily for queued attendance events.

This is desirable because students may form a queue at the attendance point. The design therefore supports the target of **≤3 s median verification time** and **≥12 students per minute**, while the amount of data required for offline event queuing remains comparatively small.

KISS at the Terminal, Richness at the Server

The terminal shall remain as simple as practical, while more complex processing and management shall be performed by the server.

- **Design rule:** The ESP32 shall primarily handle sensor and peripheral I/O, local feedback, timekeeping, and event queueing, while server-side software handles face recognition, database processing, and dashboard functions.

This follows the **KISS (Keep It Simple)** principle because reducing the complexity of embedded software improves reliability, testing, and debugging. It also supports the project's breadboard fallback, since the core hardware interfaces can be demonstrated without reproducing the entire server environment on the terminal.

Honest Measurement

System performance shall be presented according to evidence rather than assumption.

- **Design rule:** Every performance claim shall be traceable to a defined test, and values that have not yet been measured shall be reported as **targets** or **to be measured**.

This is desirable because unverified claims could give a misleading impression of system performance. Traceable measurements also provide a consistent basis for assessing the biometric error rates, verification time, throughput, battery life and offline operation against the stated acceptance criteria.

Development Methods

The project used an **incremental development approach organised around a vertical slice**. Rather than completing all hardware, software and documentation activities independently before integration, development began with a small end-to-end working path. The first vertical slice was an enrolled user whose fingerprint was captured by the terminal, processed by the verification logic, submitted to the Fast API server, stored as one idempotent attendance record, and displayed on the lecturer dashboard. Additional functionality was added incrementally after this basic path had been demonstrated.

Hardware development followed a **breadboard-to-PCB staging approach**. Interfaces such as the fingerprint sensor, LCD, indicators, RTC, and power circuitry were first assembled and tested on a breadboard. Once the interfaces had been shown to operate correctly, the design was transferred to the custom double-sided PCB. The breadboard implementation was retained as a demonstration fallback.

Software and hardware were tested continuously during development rather than being left until a single final testing phase. Each increment therefore included implementation, integration and verification activities. This approach was particularly appropriate because biometric performance, verification thresholds and end-to-end response times require measured results and may require adjustment during development.

Several established development methods were considered.

Waterfall

The Waterfall model was considered because it provides a sequential development structure in which requirements, design, implementation, and testing are performed as distinct stages [3]. It was **not adopted as the primary method**.

A strictly sequential process was considered unsuitable because biometric verification requires measurement-driven iteration. For example, the fingerprint and face-recognition acceptance targets cannot simply be assumed from the initial design; they must be measured and, where necessary, the system configuration or operating conditions must be adjusted. Delaying these measurements until the end of development would increase the risk of discovering performance problems too late.

Agile/Scrum

Agile development principles were **partially adapted**. Scrum promotes iterative development and the delivery of usable increments, allowing the product to be inspected and adapted throughout development [4]. These principles supported the decision to develop the system in increments, with a working demonstration at the end of each meaningful development cycle.

However, the complete Scrum framework was not adopted because the project was completed by a **single developer**. Formal Scrum roles, such as Product Owner and Scrum Master, together with team-oriented ceremonies, were therefore not appropriate to the project's structure. The project adopted the useful principle of iterative increments without attempting to reproduce the full Scrum framework.

V-Model

The V-model was **adopted in part** because it provides a strong relationship between system specification and verification. In the V-model, requirements and design activities on the development side are associated with corresponding integration, verification, and validation activities on the other side [5].

For this project, the V-model was mainly used to ensure that each important design requirement had a planned verification activity. This principle was implemented through the **requirements traceability matrix**, allowing a requirement to be followed from its originating stakeholder need through to the system component that addresses it and the evidence used to determine whether it has been satisfied. Requirements-verification matrices are also used in systems engineering to connect uniquely identified requirements with their verification activities and evidence [6].

Hardware-software co-design and breadboard-to-PCB staging

Because the project is an embedded system, hardware and software were developed as interacting parts of one system rather than as completely independent products. Hardware/software co-design considers the interaction between hardware and software during system development and includes prototyping and verification activities [7].

The project therefore used a staged hardware-software approach:

- **Breadboard stage:** hardware interfaces were assembled and individually tested.

- **Interface integration stage:** the ESP32 firmware was integrated with the fingerprint sensor, LCD, RTC, indicators, and communication interfaces.
- **Server integration stage:** terminal data was connected to the FastAPI backend and database.
- **End-to-end stage:** the terminal, server, and dashboard were tested as one attendance system.
- **PCB stage:** the verified breadboard design was transferred to the custom double-sided PCB.

This reduced the risk of transferring an unverified circuit directly to the final PCB and made faults easier to isolate during early development.

Adopted development approach

The final methodology was therefore a **hybrid approach** rather than a single formal methodology:

Incremental vertical-slice development + selected Agile principles + V-model verification and traceability + hardware-software co-design with breadboard-to-PCB staging.

This combination was selected because it suited a single-developer embedded system in which hardware interfaces, software services and biometric performance had to be integrated progressively and verified using measurable acceptance criteria.

Requirements Traceability Matrix as a Method Artefact

The **requirements traceability matrix (RTM)** was used as a development and verification artefact. Its purpose was to ensure that each important requirement could be traced through the design to a specific verification activity and supporting evidence. This follows the V-model principle of connecting requirements with their corresponding verification activities [3], [4].

The flow of the matrix is:

Stakeholder need → Requirement ID → Architecture component → Verification method → Evidence

This provides a direct link between what the system is required to achieve, where the requirement is implemented, and how compliance will be demonstrated.

Stakeholder Need	Requirement ID	Architecture component	Verification method	Evidence
Students should be identified accurately using fingerprints.	01	Fingerprint sensor + ESP32 terminal firmware	Controlled fingerprint identification test at sensor security level 3	Fingerprint test dataset and calculated FRR
Students should also be verified using face recognition when fingerprint acquisition is unsuccessful or low confidence.	02	ESP32-CAM + Fast API face-recognition service	Controlled face-verification test under venue lighting	Face-recognition test log and calculated FNMR/FMR
Attendance should be recorded quickly enough to support a lecture class.	03	Terminal + Fast API attendance API + PostgreSQL	Measure end-to-end verification time and students processed per minute	Timestamped test log showing median/95th-percentile time and throughput
The system should prevent the same student from being	04	Fast API attendance	Attempt repeated submissions	Database query showing zero

recorded twice in one session.		service + PostgreSQL attendance table	using the same student and session identifiers	duplicate (student_id, session_id) records
Device operation must support portability.	05	Lithium-ion battery + BMS	Device should be able to operate independently from a wall outlet	Device must be able to operate for at least one session length.

The combined approach therefore provides iteration where measurement is required, while retaining explicit verification and traceability. It is suited to a single-developer embedded project because it allows hardware, firmware, and server components to be integrated progressively without requiring the formal organizational overhead of a full software-development framework.

Architectural Strategies

The architecture was designed around the project goals of verification accuracy, response time, portability, reliability, privacy, and future extensibility. The following strategies describe the major decisions that determined how the terminal, camera, server, and dashboard were organised.

1. Distribution of Computation

Decision: Face recognition was performed on the server, while the ESP32-CAM only captured and streamed JPEG frames over Wi-Fi. This kept the camera responsible for image acquisition and left the computationally intensive recognition process to the laptop/server, which has substantially greater processing and memory resources.

The alternative of performing face recognition directly on an ESP32-S3 was rejected because it would introduce additional PSRAM and memory constraints, greater sensitivity to illumination, and a potential reduction in recognition performance. For the classic ESP32 used in the terminal, an on-device vision approach would also introduce additional USB-host and processing complexity, so server-side recognition was preferred.

2. Recogniser Selection

Decision: The face-recognition service used dlib ResNet-based face embeddings, selected for their established performance and suitability for embedding-based comparison. The commonly reported LFW benchmark for the dlib model is approximately 99.38% accuracy, although this benchmark value was not treated as the project's own measured performance.

OpenCV LBPH was considered as an alternative because it is relatively lightweight and straightforward to implement, but it was rejected because its performance is more sensitive to changes in illumination and is generally less suitable for the required recognition accuracy. NIST has also reported that false-negative performance can be strongly affected by image quality, including under-exposure of dark-skinned subjects; consequently, the design uses controlled exposure during enrolment and reports FNMR by cohort rather than relying only on an overall value.

3. Matching Topology

Decision: The primary identification topology was 1:N matching against the enrolled class, with enrolment deliberately capped at approximately one class of 60 students. This provides a practical

attendance workflow because the student's fingerprint can be identified without requiring the student to enter an identifier first.

A 1:1 comparison after student-number entry was considered because it would reduce the probability of a false identification and reduce the search space, but it was rejected as the primary workflow because it introduces an additional interaction and slows attendance capture. It was therefore retained as the non-biometric fallback, where the student enters the student number, and the lecturer confirms the identity.

4. Persistence and Distribution

Decision: PostgreSQL was used as the authoritative server-side database, while the terminal maintained a local queue for attendance events when network connectivity was unavailable. Queued events were protected using timestamps, nonces, HMAC authentication, and idempotency keys before synchronisation with the server.

An online-only architecture was rejected because reliable operation cannot be assumed on campus networks, particularly during network interruptions or load-shedding-related infrastructure disruptions. The local queue therefore trades some implementation complexity and storage requirements for improved reliability and allows queued events to be synchronised after reconnection.

5. Error Detection and Recovery

Decision: The system used fail-closed signalling, bounded retries with exponential backoff, idempotency keys, and a lecturer-controlled manual override. A failed or uncertain biometric verification therefore does not silently produce a successful attendance record.

A design in which every failure was simply retried indefinitely was rejected because it could block the attendance workflow and create excessive network requests. Likewise, automatic acceptance after repeated biometric failure was rejected because reliability had to be balanced against the risk of incorrect attendance attribution; manual override was instead retained with an append-only audit record.

6. Interface Paradigm

Decision: The terminal provided immediate local feedback through a **16×2 LCD, LEDs, and a buzzer**, while the browser-based dashboard was used for session management, monitoring, and administrative functions. This separates the student's interaction from the lecturer's management interface and allows the terminal to provide feedback without depending on a browser or phone.

A terminal-only graphical interface was rejected because it would increase hardware complexity and would not be practical for managing sessions, enrolment, and attendance records. Conversely, a dashboard-only feedback model was rejected because students require immediate confirmation of whether their attendance attempt was accepted, rejected, or requires another action.

7. Extensibility

Decision: The architecture included a device registry so that additional attendance terminals could be incorporated without redesigning the server's core attendance model. The current implementation nevertheless remained focused on the single-class prototype and its defined attendance use case.

More extensive capabilities, including institutional-system integration, LMS integration, and access-control actuation, were considered but deferred to future work because they would introduce additional interfaces, security requirements, and data-management complexity beyond the project's primary objective. This preserves a clear prototype boundary while keeping the architecture extensible enough for later development.

SYSTEM ARCHITECTURE

The system was organised as a three-tier biometric attendance architecture consisting of the terminal, Dashboard, and Backend tiers**.** The decomposition separated hardware-dependent biometric acquisition and immediate student interaction from computationally intensive face recognition and central data management.

This structure was selected to achieve four main design goals:

- independent testing of the embedded hardware;
- centralized processing and data management;
- reliable attendance captured during temporary network failures; and
- clear interfaces between subsystems to simplify integration and future extension.

Major System Responsibilities

At the highest level, the system was required to perform the following responsibilities:

- **Enrolment:** associate each learner with the biometric information required for later attendance identification.
- **Session management:** allow the lecturer or administrator to open an attendance session and provide the course and venue information.
- **Biometric acquisition:** capture fingerprint information at the terminal and facial image frames through the camera.
- **Identity matching:** perform primary 1 fingerprint identification and secondary face recognition when required.
- **Attendance decision:** determine whether the available verification result is sufficient to record attendance.
- **Durable recording:** store attendance events in the server database.
- **Retrieval and reporting:** allow authorised users to view and retrieve attendance information through the dashboard.
- **Audit:** record enrolment, deletion, manual override, and export activities.
- **Privacy lifecycle:** protect biometric information and apply the defined retention and deletion rules.

Three-Tier Decomposition

The responsibilities were divided between the three architectural tiers.

Tier	Primary Responsibilities
Terminal	Fingerprint acquisition and local 1 matching, student feedback, RTC timekeeping, offline event queueing, and secure communication. Also, JPEG image acquisition and transmission to the server; no on-chip recognition
Backend	Session management, face recognition, verification coordination, durable storage, attendance retrieval/reporting, audit and privacy lifecycle
Dashboard	The lecturer sets the sessions, and administration will monitor and track the data.

This decomposition ensured that each tier had a clearly defined role rather than combining all system functions in a single device.

Attendance Terminal

The attendance terminal was the main physical point of interaction with the learner. It combined fingerprint and facial-image acquisition into a single portable device.

The terminal contained:

- ESP32 controller;
- R307/AS608-class optical fingerprint module;
- ESP32-CAM;
- 16×2 LCD;
- LED and buzzer indicators;
- battery-backed RTC; and
- rechargeable Li-ion power subsystem.

Its responsibilities were to:

- acquire the learner's fingerprint;
- obtain the local fingerprint match result;
- capture facial frames when secondary verification was required;
- provide immediate learner feedback;
- maintain local device time;
- queue attendance events during temporary network failure; and
- securely communicate with the server.

The terminal was deliberately isolated from the server-side recognition and database functions. This allowed the hardware to be tested independently and ensured that the **breadboard implementation could remain a functional fallback** while the custom PCB was being developed or debugged.

[Insert Figure 3 here: Block diagram of the terminal]

Backend

The Backend provided the central processing and data-management functions.

It consisted of:

- **FastAPI backend** – application logic and communication coordination;
- **Python face-recognition service** – secondary face verification using dlib ResNet embeddings; and
- **PostgreSQL database** – persistent system and attendance data storage.

The server was responsible for:

- receiving authenticated terminal requests;
- coordinating biometric verification;
- performing secondary face recognition;
- applying the attendance decision and fusion logic;
- maintaining attendance records;
- managing queued-event synchronisation;
- maintaining the audit trail; and
- enforcing biometric privacy and retention controls.

Face recognition was centralised on the server because the required **computational resources, model files, and encrypted face-template/embedding store** were located there. This avoided placing the recognition workload and protected biometric storage on the embedded terminal.

Dashboard

The dashboard provided the management interface for lecturers and administrators.

Its responsibilities included:

- creating and managing attendance sessions;
- entering course and venue metadata;
- supporting authorised enrolment;
- viewing attendance records;
- retrieving attendance information for reporting; and
- performing authorised administrative operations.

The dashboard did not access PostgreSQL directly. Instead, it communicated with the Backend API, which controlled authentication, authorisation, and application logic.

This separation reduced the coupling between the user interface and the database and allowed the backend rules to remain consistent regardless of how the dashboard was presented.

Biometric Processing and Decision Flow

Fingerprint was the **primary biometric modality**. The R307 performed the local fingerprint matching against the enrolled class and returned the match result to the ESP32.

Face recognition was the **secondary or confirmatory modality**. It was invoked when:

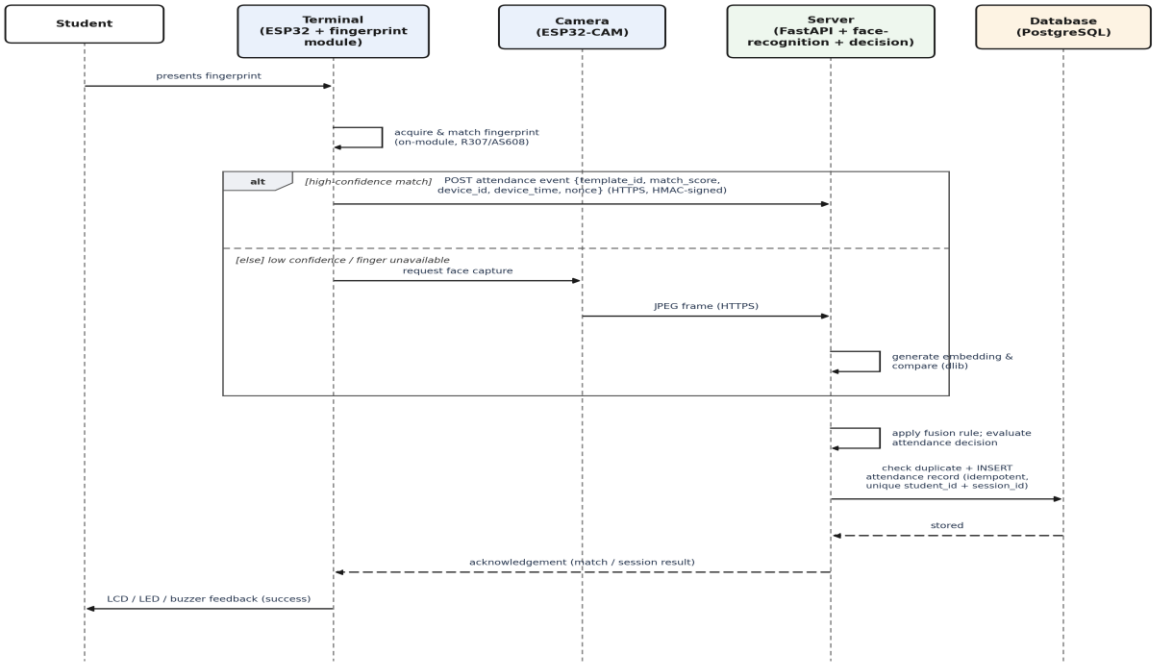
- the fingerprint could not be acquired; or
- the fingerprint result was below the defined confidence condition.

The overall attendance process therefore involved cooperation between the terminal and server:

Fingerprint acquisition → Local fingerprint matching → Result sent to server → Face verification when required → Attendance decision → Durable recording

The normal successful workflow is represented by **Figure 4: Sequence diagram - successful attendance capture**.

Figure 4 – Sequence Diagram: Successful Attendance Capture



Offline Operation

The architecture was designed to continue capturing attendance events during temporary network interruptions.

When the server was unavailable:

1. The terminal captured and validated the attendance event.
2. The event was placed in the terminal's local queue.
3. The terminal retained the device timestamp and synchronisation information.
4. After reconnection, the queue manager passed the event to the network client.
5. The event was transmitted using the defined HMAC and idempotency mechanisms.
6. The server validated the event and stored it in PostgreSQL.

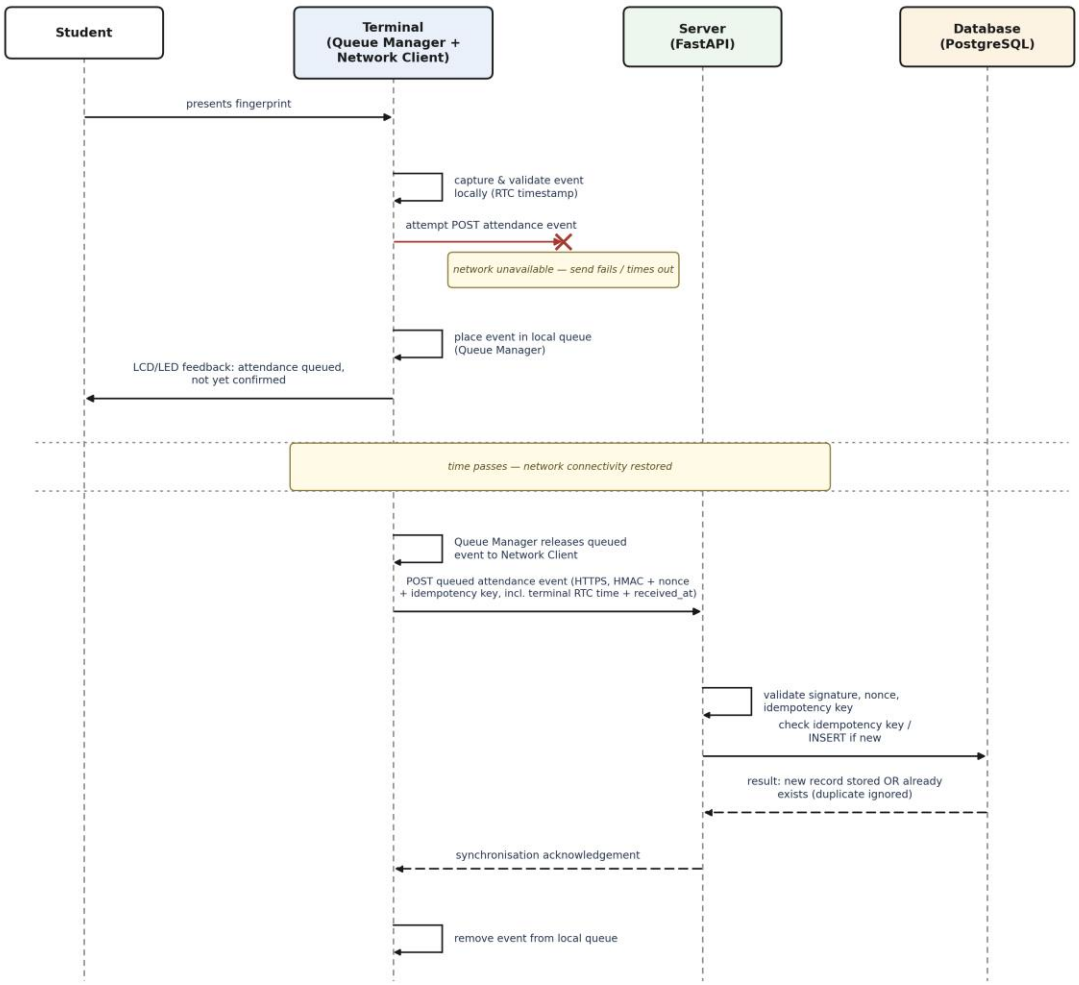
The resulting flow was:

Capture → Local queue → Reconnection → Authenticated synchronisation → Database

This approach was preferred to an online-only architecture because campus Wi-Fi and network availability could not be assumed to be continuous.

The complete offline process is represented by **a Sequence diagram: offline capture and deferred synchronisation**.

Figure 5 – Sequence Diagram: Offline Capture and Deferred Synchronisation



The architecture was selected around **testability, resource allocation, reliability, and maintainability**.

The terminal was kept as an independent physical unit so that its hardware interfaces could be tested without requiring the complete server environment. This was particularly important because development followed a **breadboard-first approach**, with the custom PCB fabricated only after the major interfaces had been demonstrated.

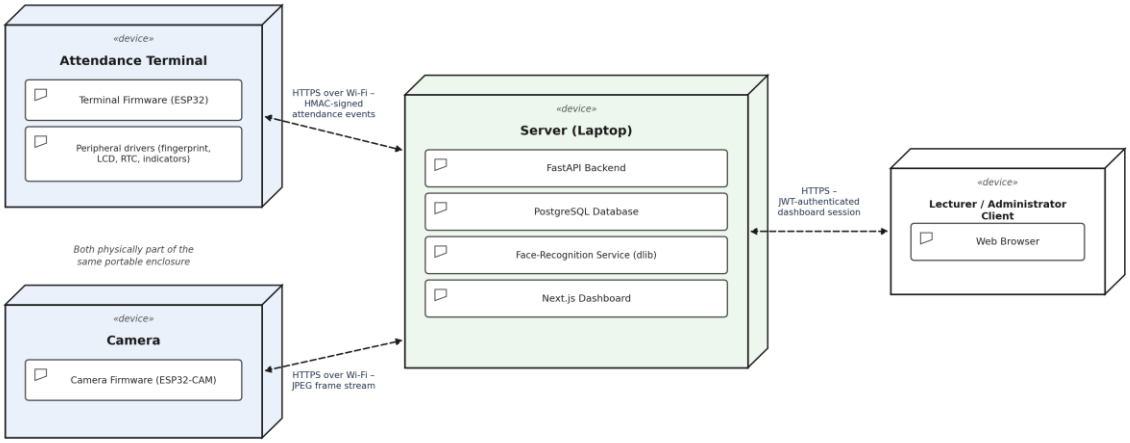
Recognition was centralised because face processing required more computing resources than were appropriate for the embedded terminal. Centralising the recognition service also placed the model files and encrypted biometric embedding store in one controlled environment.

The camera was logically separated from the recognition service even though it formed part of the same physical terminal. This allowed the **ESP32-CAM to be replaced by a laptop webcam during early testing without changing the recognition API**. As a result, server-side recognition could be developed and tested before final camera integration.

The dashboard was separated from the server because presentation and application logic had different responsibilities. This allowed the same backend services and security rules to support the dashboard without exposing the database directly to the browser.

The complete physical and logical deployment relationship is shown in **the Deployment diagram**.

Figure 2 – Deployment Diagram



Overall, the architecture followed a **separation-of-concerns and layered-processing approach**:

Attendance Terminal → Server → Dashboard

with the server providing the central coordination point between physical attendance capture, biometric recognition, persistence, and management functions.

SUBSYSTEM ARCHITECTURE

The attendance terminal was internally decomposed into several firmware functions. These functions were coordinated by a **state machine** so that acquisition, user feedback, communication, queueing, and time management remained separate but could operate as one attendance workflow.

Acquisition Task

The acquisition task was responsible for biometric input.

Its main functions were:

- communicating with the fingerprint module;
- initiating fingerprint acquisition;
- receiving the local match result; and
- passing the result to the control logic.

The task did not directly manage database storage or dashboard operations.

HMI Task

The HMI task was responsible for immediate learner feedback.

It controlled:

- the **LCD**;
- status **LEDs**; and
- the **buzzer**.

The HMI therefore translated the internal terminal state into a simple indication such as waiting, successful verification, failed verification, or a communication problem.

Queue Manager

The queue manager provided local **store-and-forward** functionality.

It was responsible for:

- accepting attendance events that could not immediately be transmitted;
- retaining the events locally;
- tracking events awaiting synchronisation; and
- releasing queued events when the network became available.

This function supported the requirement for continued operation during temporary network failures.

Network Client

The network client handled communication between the ESP32 and the FastAPI server.

It was responsible for:

- constructing the attendance payload;
- applying HMAC authentication;
- sending HTTPS requests;
- detecting communication failures;
- performing bounded retries; and
- synchronising queued events after reconnection.

The network client therefore acted as the communication boundary between the embedded terminal and the server.

RTC/Time Manager

The RTC/time manager maintained the terminal's local time.

It provided:

- the terminal's device timestamp;
- timing information for offline events; and
- synchronisation-related time information.

The **server remained responsible for the canonical attendance timestamp**. The terminal time was therefore supporting information rather than the authoritative attendance time.

State Machine

The state machine linked the firmware functions and controlled the overall terminal workflow.

The high-level sequence was:

Idle
↓
Fingerprint acquisition
↓
Match evaluation
↓
Secondary verification, when required
↓
Attendance decision
↓
Queue / transmit
↓
Confirmation
↓
Idle

The state machine also controlled error conditions. A failed biometric verification, unavailable network, or unsuccessful communication did not result in an automatic successful attendance record. Instead, the terminal entered the appropriate failure or recovery state.

This internal decomposition allowed the major firmware responsibilities to remain separate while still providing one controlled attendance-capture process. Detailed circuit design, hardware components, and firmware implementation are discussed in the subsequent detailed design sections.

DETAILED SYSTEM DESIGN

The detailed system design decomposed the system architecture into five principal components: **terminal firmware, fingerprint subsystem, face-recognition service, attendance API, and data store**. Each component was described according to the defined software-component attributes so that its purpose, responsibilities, constraints, internal composition, and externally available interfaces could be traced to the higher-level architecture.

Terminal Firmware

Classification
The terminal firmware was classified as an **embedded application** executing on the ESP32 microcontroller. It formed the control layer between the fingerprint subsystem, learner-facing hardware, local storage, RTC, and server communication interface.

Definition
The terminal firmware implemented the control logic required for the portable attendance terminal. It coordinated biometric acquisition, user feedback, local timing, attendance-event preparation, temporary storage, and secure communication with the server.
The firmware did not implement server-side face recognition or durable attendance storage. Its role was to operate the physical terminal and provide a reliable communication boundary to the server.

Responsibilities
The terminal firmware was responsible for:
- controlling the fingerprint subsystem;
- receiving and interpreting fingerprint match results;
- controlling the LCD, LEDs, and buzzer;
- obtaining the terminal's RTC time;

- creating attendance-event payloads;
- maintaining the offline event queue;
- communicating with the Attendance API;
- applying HMAC authentication to outgoing requests;
- detecting communication failure and initiating bounded retries;
- synchronising queued events after reconnection; and
- maintaining the terminal state machine.

The firmware also supported the privacy and security requirements by ensuring that raw fingerprint images and fingerprint templates were not transmitted to the server.

Constraints

The firmware operated within the resource limitations of the ESP32 and therefore avoided unnecessary local computation and storage.

The main constraints were:

- the terminal was required to support a **1 class-sized identification workflow**;
- fingerprint performance was required to meet the target **FRR ≤ 2% at security level 3**, with **FAR < 0.001%** as the sensor target;
- end-to-end verification was targeted at **≤ 3 s median and ≤ 5 s at the 95th percentile**;
- attendance throughput was targeted at **≥ 12 students/minute**;
- the offline queue was required to hold **≥ 500 events**;
- queued events were required to synchronise within **60 s of reconnection**;
- battery operation was targeted at **≥ 2 hours**;
- outgoing network messages had to contain the required device metadata and authentication information;
- retries were bounded and could not continue indefinitely;
- successful attendance could not be generated solely from a communication failure;
- duplicate synchronisation was required to remain idempotent; and
- the firmware had to remain usable with the breadboard implementation as a development and demonstration fallback.
-

Composition

The terminal firmware was divided into cooperating functional units:

- **Acquisition task** – controls biometric input and receives the fingerprint result.
- **HMI task** – controls the LCD, LEDs, and buzzer.
- **Queue manager** – stores events that cannot immediately reach the server.
- **Network client** – manages authenticated HTTPS communication.
- **RTC/time manager** – maintains the terminal's local time.
- **State machine** – coordinates the activities and controls system transitions.

The ESP32-CAM formed part of the same physical attendance terminal but remained logically separated from the fingerprint-control functions because its primary responsibility was image acquisition.

Resources

The firmware used and managed the following resources:

- ESP32 processor and RAM;
- non-volatile/local queue storage;
- UART connection to the fingerprint module;
- I2C connection to the LCD and RTC;
- GPIO resources for indicators;
- Wi-Fi networking;
- rechargeable battery power; and
- the device-specific shared secret used for message authentication.

Resource contention was minimised by assigning each peripheral to a defined firmware function. The design avoided blocking operations that could prevent the network client, queue manager or HMI from responding. Shared event data was therefore passed between logical functions under controlled state transitions rather than allowing unrestricted concurrent modification.

Processing

The firmware operated as a state-driven process. After initialisation, the terminal entered an idle state and waited for an attendance interaction.
The general processing sequence was:
Initialisation → Idle → Fingerprint acquisition → Local match evaluation → Secondary verification request, when required → Attendance decision → Queue/transmit → Feedback → Idle

For normal online operation, a successful fingerprint result was packaged with the required metadata and sent to the server. When secondary face verification was required, the camera path was activated, and the resulting frame stream was handled by the server-side recognition service.
When communication failed, the event was written to the local queue instead of being discarded. The queue manager retained the event until reconnection, after which the network client attempted authenticated synchronisation.
Retries used bounded exponential backoff. This prevented continuous retry traffic and allowed the terminal to return to a usable state during prolonged connectivity failures.
The firmware also maintained the device timestamp using the RTC. The terminal timestamp was retained for event traceability, while the server remained responsible for the canonical attendance timestamp.
Detailed implementation logic is provided in the **annotated terminal-firmware listings in the appendices**.

Fingerprint Subsystem

Classification
The fingerprint subsystem was classified as a **hardware module plus driver**. It consisted of the R307/AS608-class optical fingerprint sensor and the embedded driver responsible for UART communication and command handling.

Definition
The fingerprint subsystem provided the terminal's **primary biometric modality**. It performed fingerprint enrolment and local fingerprint matching, allowing a learner to be identified against the enrolled class without transmitting raw fingerprint information to the server.

Responsibilities
The subsystem was responsible for:
- fingerprint image acquisition at the sensor;
- fingerprint enrolment;
- template creation and storage within the sensor;
- 1 identification against the enrolled class;
- returning a match identifier and associated score/status;
- reporting acquisition or matching failures; and
- communicating these results to the ESP32 driver.

The subsystem therefore provided the primary identity result used by the attendance decision process.

Constraints
The fingerprint subsystem was required to operate at **security level 3** for the acceptance testing configuration.
Its acceptance targets were:
- **FRR ≤ 2%**;
- **FAR < 0.001%**;
- class-sized enrolment limited to approximately **60 learners**; and
- support for the defined attendance throughput target of **≥ 12 students/minute** at system level.

The module was also subject to the requirement that raw fingerprint images and templates did not traverse the network.
The deliberately class-sized enrolment limit was an architectural constraint rather than an indication that the sensor could theoretically support only that number of entries. The limit

reduced the cumulative 1 false-accept risk and kept the prototype within its defined use case.

Composition
The subsystem consisted of:
- optical fingerprint sensing hardware;
- the sensor's internal fingerprint processing and template store;
- UART communication;
- the ESP32-side driver; and
- error/status handling in the driver.

The ESP32 interacted with the module through its defined command and response interface rather than directly manipulating fingerprint-processing algorithms.

Resources
The fingerprint subsystem required:
- sensor power;
- UART pins;
- serial communication bandwidth;
- the sensor's internal template memory; and
- physical access to the learner's finger.

The sensor was the touch-based biometric component of the system. Hygiene considerations therefore required a wipeable enclosure and cleaning by campus sanitation staff.

Processing
The subsystem followed the sensor's standard acquisition and matching sequence.
During enrolment, the finger was presented, and the module acquired the required fingerprint data before generating and storing the corresponding template.
During attendance:
1. the driver requested a fingerprint capture;
2. the sensor acquired the fingerprint;
3. the sensor compared the captured fingerprint against enrolled templates;
4. the sensor returned the identified template identifier and match information; and
5. the ESP32 passed the result to the terminal control logic.

When a usable fingerprint could not be acquired, or the match result was below the defined confidence condition, the terminal initiated the secondary face-recognition path.
The sensor's internal processing meant that raw fingerprint images and templates remained within the fingerprint subsystem and were not transmitted through the server interface.

Face Recognition Service
Classification

The face recognition service was classified as a **Python service module** operating on the server.

Definition

The service provided the **secondary/confirmatory biometric modality**. It received facial image frames from the attendance workflow, generated facial embeddings, and compared them against the authorised class enrolment set.

The service performed recognition centrally rather than on the ESP32-CAM.

Responsibilities

The face-recognition service was responsible for:

- receiving facial images through the server-side processing path;
- detecting and preparing faces for recognition;

- generating dlib ResNet-based face embeddings;
- comparing an input embedding with an authorized enrolled embedding;
- returning the best matching identity and comparison result;
- reporting failed or ambiguous recognition; and
- supporting the project's FNMR/FMR measurement process.

The service was not responsible for attendance persistence, session management, or dashboard presentation.

Constraints

The principal target was:

- **FNMR ≤ 5% at FMR 1e-3 under venue lighting**.

The face-recognition service was also constrained by image quality and environmental conditions. Illumination could affect recognition performance. To reduce unnecessary variation during enrolment, the design specified **controlled/fixed camera exposure conditions** where practical.

The system was required to report FNMR by cohort during evaluation rather than relying only on a single aggregate value, because recognition performance can vary with image quality and subject conditions.

The face service operated as a **secondary verification mechanism**, not as an independent replacement for the fingerprint subsystem.

Composition

The service consisted of:

- image input handling;
- face detection and preprocessing;
- dlib ResNet embedding generation;
- encrypted embedding retrieval;
- embedding comparison;
- threshold/evaluation logic; and
- result formatting.

The encrypted face embeddings were stored separately from attendance records, with a pseudonymous internal relationship to the student identity.

Resources

The service used:

- server CPU and RAM;
- Python runtime and required recognition libraries;
- dlib ResNet model files;
- image-processing resources;
- protected face-embedding storage; and
- the server-side API interface.

The model files and biometric embedding store were intentionally located on the server rather than the terminal.

Processing

The service processed a supplied facial frame through the following high-level sequence:

Frame acquisition → Face detection → Image preparation → Embedding generation → Protected embedding retrieval → Similarity comparison → Threshold evaluation → Recognition result

The service returned a result to the API rather than directly modifying the attendance database.

Where no sufficiently reliable match was obtained, the service returned an unsuccessful or inconclusive result. It did not convert an uncertain biometric result into a successful identity automatically.

The recognition process was evaluated against the specified **FNMR ≤ 5% at FMR 1e-3** target. These values were treated as acceptance criteria and not as achieved results until measured during validation.

> Presentation-attack evaluation followed the project requirement to report **APCER and BPCER** for printed-photograph, phone-screen replay, and fingerprint-lift attacks, with an optional blink/head-turn challenge.

Attendance API

Classification

The Attendance API was classified as a **FastAPI application**. It formed the principal application and communication layer between the attendance terminal, face-recognition service, dashboard, and PostgreSQL database.

Definition

The API provided authenticated services for attendance capture, synchronisation, session management, enrolment support, and retrieval of attendance information.

It also enforced server-side validation and ensured that terminal requests could not directly manipulate the database without passing through the application rules.

Responsibilities

The Attendance API was responsible for:

- authenticating attendance terminals;
- validating HMAC-protected device requests;
- managing attendance sessions;
- coordinating primary and secondary biometric verification;
- invoking the face-recognition service when required;
- applying the defined attendance decision/fusion rule;
- writing durable attendance records;
- preventing duplicate attendance records;
- receiving and synchronizing offline events;
- supporting dashboard operations;
- enforcing role-based access;
- recording audit events; and
- enforcing data-access rules associated with biometric privacy.

Constraints

The API was subject to the following acceptance and security constraints:

- end-to-end verification target: **≤ 3 s median and ≤ 5 s at the 95th percentile**;

- throughput target: **≥ 12 students/minute**;
- exactly one attendance record per (student_id, session_id);
- idempotent processing of retried queued events;
- HMAC authentication for terminal-generated requests;
- nonce and timestamp validation to prevent replay;
- HTTPS for network communication;
- short-lived JWT authentication for dashboard access;
- role-based access for student, lecturer, and administrator functions; and
- manual override actions recorded in the audit log.

The API did not accept raw fingerprint templates or fingerprint images from the terminal.

Composition

The FastAPI application was composed conceptually of:

- authentication and authorization layer;
- session-management routes;
- attendance-capture routes;
- synchronization routes;
- enrolment-related routes;
- face-recognition service integration;
- validation and decision logic;
- PostgreSQL data-access layer; and
- audit logging functions.

The API therefore acted as the orchestration layer rather than embedding all recognition functions directly in every endpoint.

Resources

The API used:

- server CPU and RAM;
- FastAPI and Python runtime resources;
- PostgreSQL connections;
- face-recognition service resources;
- device shared secrets;
- JWT credentials;
- network sockets; and
- database transaction locks.

Database writes were performed using transactions so that attendance recording and duplicate protection remained consistent. The unique constraint on (student_id, session_id) provided database-level enforcement even if the same request were submitted more than once.

Processing

The principal API workflow was:

Receive request → Authenticate → Validate request → Determine session/device context → Perform or request verification → Apply decision rule → Persist result → Return response

For terminal-originated events, the API first validated the device identity, timestamp, nonce, and HMAC signature. Invalid or replayed requests were rejected.

For normal attendance, the server considered the fingerprint result supplied by the terminal and invoked the face-recognition service when the defined secondary-verification condition occurred.

The decision process preserved the project's biometric hierarchy:

- fingerprint remained the **primary modality**;
- face recognition was **secondary/confirmatory**;
- failure to acquire a fingerprint or a low-confidence fingerprint result caused the secondary path to be considered; and
- A biometric result was not converted into an attendance record unless the defined decision conditions were satisfied.

For offline synchronization, the API processed the terminal event using the supplied idempotency key. A previously processed event returned the existing result rather than creating another attendance record.

Functional Flow

The Biometric Student Attendance and Verification System consists of five main functions. These functions describe the operational behaviour of the system from student enrolment through attendance capture and finally attendance-record administration.

F1 — Student Enrolment
Student enrolment creates the identity information required for attendance identification. The process captures the student's consent, enrols the fingerprint and face where permitted, stores the resulting biometric data securely, and registers a non-biometric fallback.

F2 — Session Open and Close
The lecturer creates an attendance session by providing the course and venue information. The server assigns the canonical session time and keeps the session active until the lecturer closes it.

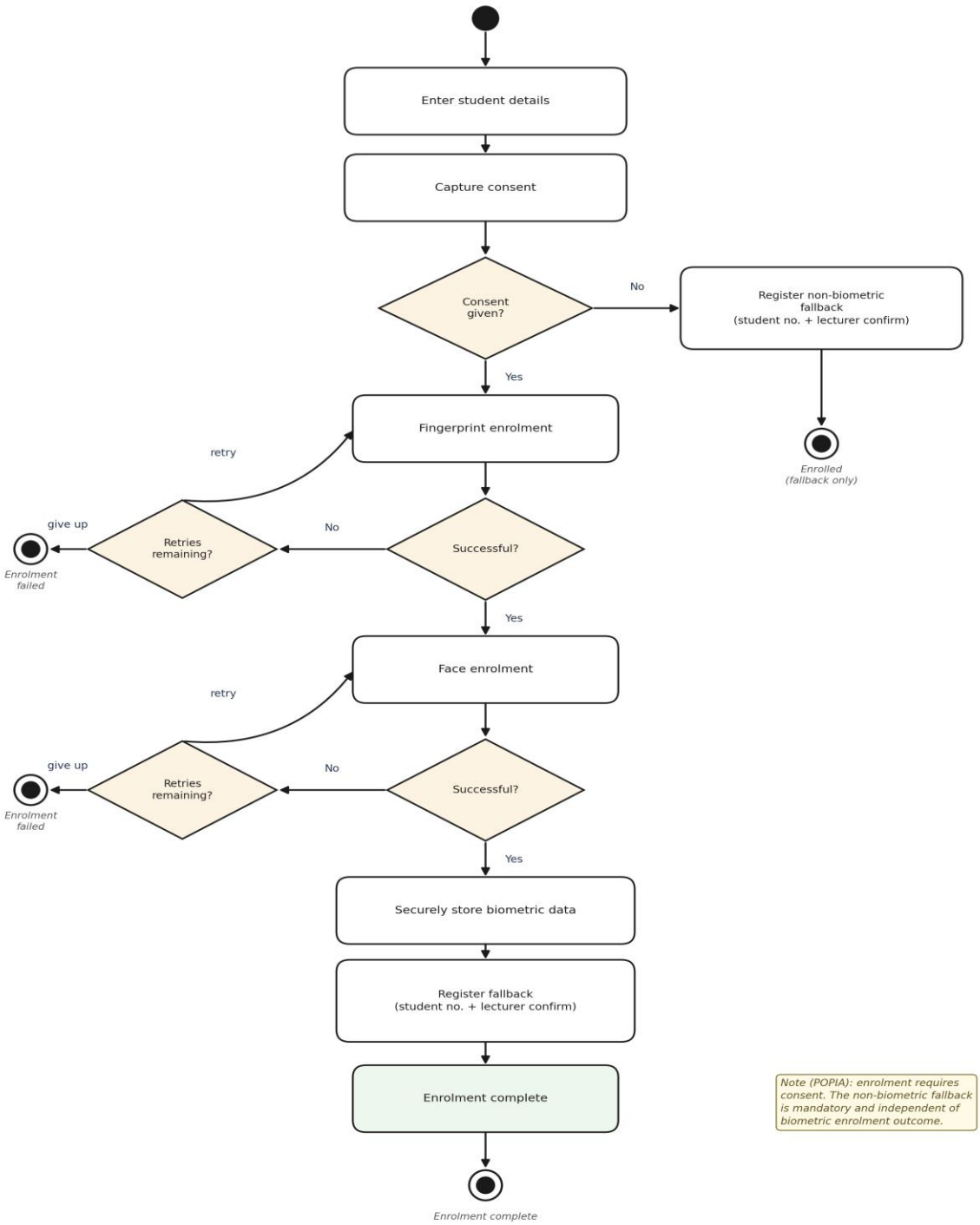
F3 — Online Attendance Capture
The student is identified at the terminal using fingerprint as the primary biometric. A high-confidence fingerprint match verifies the student directly; a low-confidence result or failed fingerprint acquisition invokes face recognition as the secondary confirmation. The verified event is then sent to the server and recorded.

F4 — Offline Attendance Capture and Synchronisation
When the server cannot be reached, the terminal continues capturing attendance and stores events locally. Once connectivity is restored, queued events are transmitted to the server and synchronised without creating duplicate attendance records.

F5 — Record Retrieval, Correction and Export
The lecturer or administrator retrieves attendance records through the dashboard, performs authorised corrections where necessary, and exports attendance information. Corrections and manual overrides are recorded in the audit trail.

F1 STUDENT ENROLMENT

Figure F1 – Activity Diagram: Student Enrolment



F1 Process Flow Table

Process ID	Process name	Description	Actor	Precondition	Postcondition
F1-P1	Capture student information	The student's number and required class information are recorded.	Lecturer/Administrator	Students are eligible for enrolment.	Student identity record exists.

F1-P2	Capture consent	The student's consent for biometric processing is recorded before biometric collection.	Student	Student information has been entered.	Consent decision is recorded.
F1-P3	Enroll biometrics	The fingerprint and face biometrics are captured and processed where consent and approval conditions permit.	Student/System	Valid consent has been obtained.	Fingerprint and face biometric data are available for storage.
F1-P4	Store biometric data	Fingerprint and face data are stored using defined secure storage mechanisms.	System	Biometric enrolment succeeded.	Biometric data are associated with the student identity.
F1-P5	Register fallback	A non-biometric student-number verification method is registered for cases where biometric verification cannot be used.	Lecturer/Administrator	Student identity exists.	Fallback method is available.

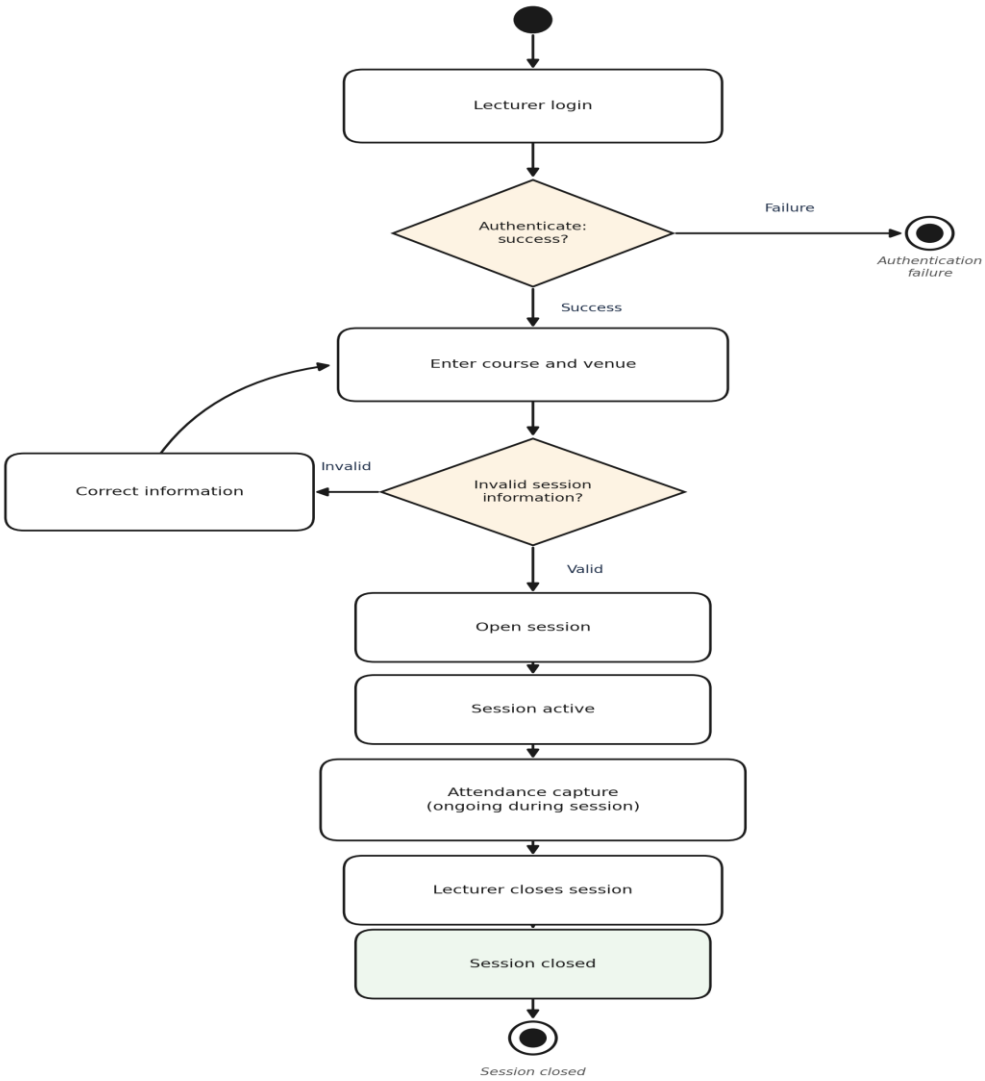
F1 Status Flow Table

Status	Meaning	Triggered by	User feedback	Next status
IDLE	Terminal is ready for an enrolment operation.	System ready.	Ready message / ready LED/none	AWAITING_FINGER
AWAITING_FINGER	Terminal is waiting for a fingerprint sample.	Fingerprint enrolment started.	“Place finger” / waiting LED/none	CAPTURING
CAPTURING	Fingerprints are being acquired or processed.	Finger detected.	Processing message/activity LED / short beep	AWAITING_FINGER or ENROLMENT_FAILED
ENROLMENT_FAILED	Required biometric enrolment could not be completed.	Failed capture or processing.	Failure message/error LED/error beep	AWAITING_FINGER or FALLBACK_MANUAL

FALLBACK_MANUAL	Students are registered for non-biometric verification.	Consent declined or biometric enrolment unavailable.	Manual-verification message/warning LED/beep	IDLE
ERROR	A system fault prevents enrolment from continuing.	Hardware, storage, or software failure.	Error message/fault LED/error beep	IDLE

F2 SESSION OPEN AND CLOSE

Figure F2 – Activity Diagram: Session Open and Close



F2 Process Flow Table

Process ID	Process name	Description	Actor	Precondition	Postcondition

F2-P1	Lecturer authentication	The lecturer logs into the dashboard and is verified.	Lecturer/System	Dashboard is available.	Lecturer is authenticated.
F2-P2	Open session	The lecturer enters the course and venue information and opens the session.	Lecturer	Lecturer is authorized.	An active attendance session exists.
F2-P3	Run attendance session	The terminal accepts attendance events for the active session.	Student/System	Session is open.	Attendance can be captured.
F2-P4	Close session	The lecturer closes the active session.	Lecturer	Session is open.	Session is closed, and new attendance is no longer accepted.

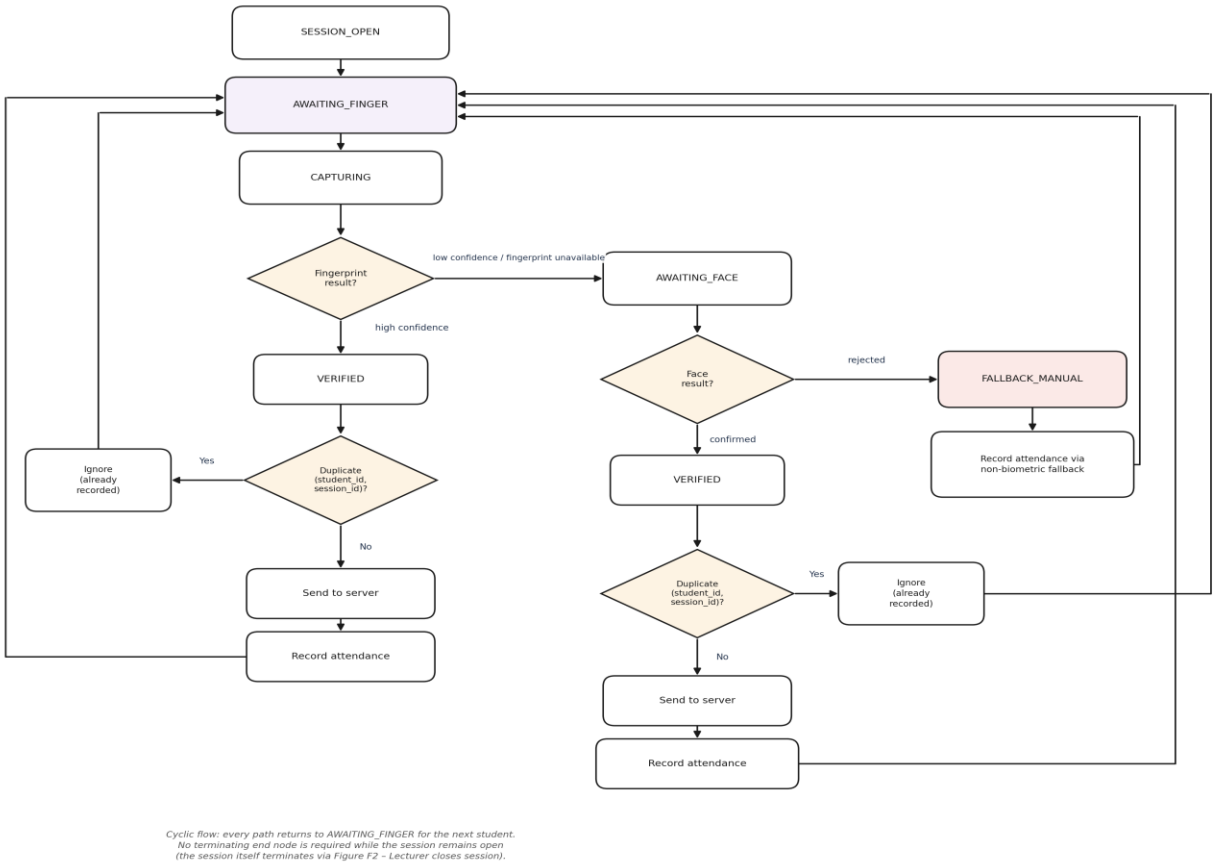
F2 Status Flow Table

Status	Meaning	Triggered by	User feedback	Next status
IDLE	No attendance session is active.	System startup or previous session closure.	“No session” / ready LED/none	SESSION_OPEN
SESSION_OPEN	A valid attendance session is active.	Lecturer opens session successfully.	Session information / active LED/confirmation beep	AWAITING_FINGER
ERROR	Session operation could not be completed.	Authentication, server, or session error.	Error message/fault LED/error beep	IDLE

F3 ATTENDANCE CAPTURE — ONLINE PATH

The online path is the normal attendance process. The fingerprint module performs the primary 1:N identification. **High-confidence fingerprint matches are accepted directly. Low-confidence fingerprint results or unsuccessful fingerprint acquisition cause the system to use face recognition as the secondary confirmation method.** This is an important security trade-off: the secondary modality improves usability and recovery, but it does not provide the same assurance as requiring two independently successful biometric matches.

Figure F3 – Activity Diagram: Online Attendance Capture



F3 Process Flow Table

Process ID	Process name	Description	Actor	Precondition	Postcondition
F3-P1	Capture fingerprint	The terminal captures the student's fingerprint and performs 1: N identification against the enrolled class.	Student/Terminal	Session is open.	Fingerprint result is available.
F3-P2	Evaluate fingerprint result	The system determines whether the fingerprint result is high-confidence or requires secondary verification.	Terminal	Fingerprint results available.	Students are verified or sent to face verification.
F3-P3	Face confirmation	The camera and face-recognition service are used when fingerprint verification is inconclusive or unavailable.	Student/System	Secondary verification required.	Face result confirms or rejects the identity.

F3-P4	Submit attendance	The terminal sends the verification event securely to the FastAPI server.	Terminal	Identity has been accepted.	Server receives the attendance event.
F3-P5	Check and record attendance.	The server validates the event, checks for duplicates, and creates the attendance record.	Server	Valid event received.	Attendance is recorded, or the duplicate is ignored.
F3-P6	Manual fallback	A lecturer confirms the student's biometric verification cannot be completed.	Lecturer	Biometric verification unsuccessful.	Attendance may be confirmed manually.

F3 Status Flow Table

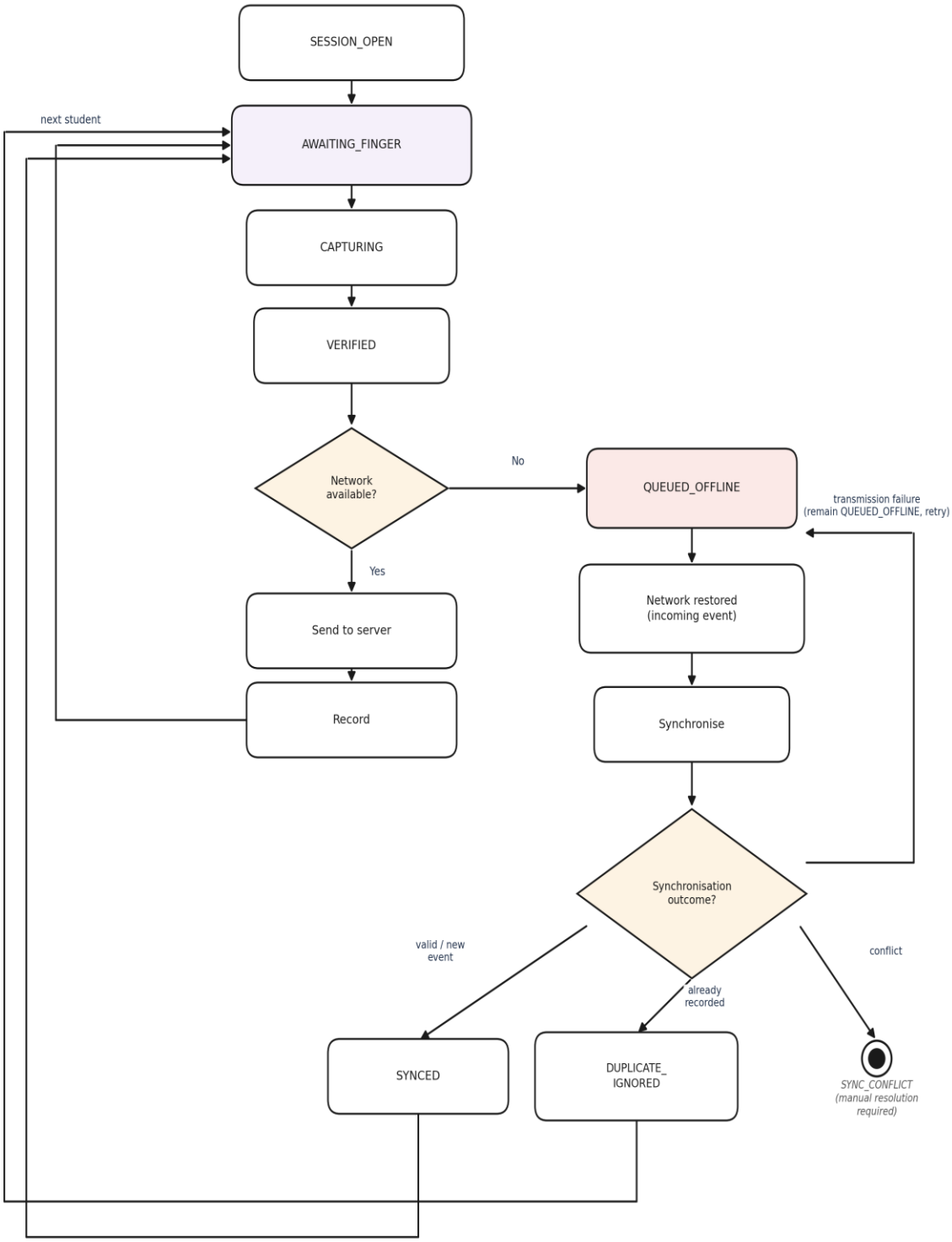
Status	Meaning	Triggered by	User feedback	Next status
AWAITING_FINGER	Terminal is waiting for a student to present a finger.	Session is active.	"Place finger" / waiting LED/none	CAPTURING
CAPTURING	Fingerprints are being captured and matched.	Finger detected.	"Processing" / activity LED / short beep	MATCH_HIGH_CONFIDENCE or MATCH_LOW_CONFIDENCE
MATCH_HIGH_CONFIDENCE	Fingerprint meets the direct-verification threshold.	High-confidence match.	"Fingerprint matched" / success LED/confirmation beep	VERIFIED
MATCH_LOW_CONFIDENCE	Fingerprint result is insufficient for direct verification.	Low-confidence match.	"Use face verification" / activity LED / prompt beep	AWAITING_FACE
AWAITING_FACE	System is waiting for a face image for confirmation.	Secondary verification required.	"Look at camera" / activity LED / prompt beep	FACE_CONFIRMED or FACE_REJECTED
FACE_CONFIRMED	Face recognition confirms the identity.	Face result accepted.	"Confirmed" / success LED/confirmation beep	VERIFIED
FACE_REJECTED	Face recognition does not confirm the identity.	Face result rejected.	"Verification failed" / warning LED/error beep	FALLBACK_MANUAL
VERIFIED	Identity has been successfully verified, and the attendance event is accepted.	Successful biometric decision.	"Attendance recorded" / success LED/confirmation beep	AWAITING_FINGER
DUPLICATE_IGNORED	Attendance already exists for the student and session.	Duplicate check detects existing record.	"Already recorded" / warning LED / short beep	AWAITING_FINGER
FALLBACK_MANUAL	Lecturer performs non-biometric confirmation.	Biometric route fails.	"Lecturer confirmation" / warning LED / prompt beep	VERIFIED or AWAITING_FINGER

ERROR	A system or communication error prevents processing.	Hardware, API, or database failure.	Error message/fault LED/error beep	AWAITING_FINGER or IDLE

F4 ATTENDANCE CAPTURE — OFFLINE AND DEFERRED SYNCHRONISATION

The offline function prevents temporary loss of attendance data when the terminal cannot communicate with the server. The terminal stores the attendance event locally and synchronises it when the connection returns. The server remains authoritative for the received attendance record and uses the terminal RTC time as supplementary offline timing information.

Figure F4 – Activity Diagram: Offline Attendance Capture and Synchronisation



F4 Process Flow Table

Process ID	Process name	Description	Actor	Precondition	Postcondition
F4-P1	Capture attendance offline	The terminal identifies and verifies the student while the server is unavailable.	Student/Terminal	Session is active.	Valid attendance event exists locally.
F4-P2	Queue attendance event	The terminal stores the event with its required identification, timing, nonce, and idempotency information.	Terminal	Server unavailable.	Event is held in the offline queue.
F4-P3	Detect reconnection	The terminal detects that communication with the server has been restored.	Terminal	Queued events exist.	Synchronization can begin.
F4-P4	Synchronize events	Queued events are sent to the FastAPI server and validated.	Terminal/Server	Network connection restored.	Events are accepted, duplicated, or flagged as conflicts.
F4-P5	Complete synchronization	Successfully processed events are marked as synchronized and removed from the pending queue.	System	Server accepted event.	Local and server records are consistent.

F4 Status Flow Table

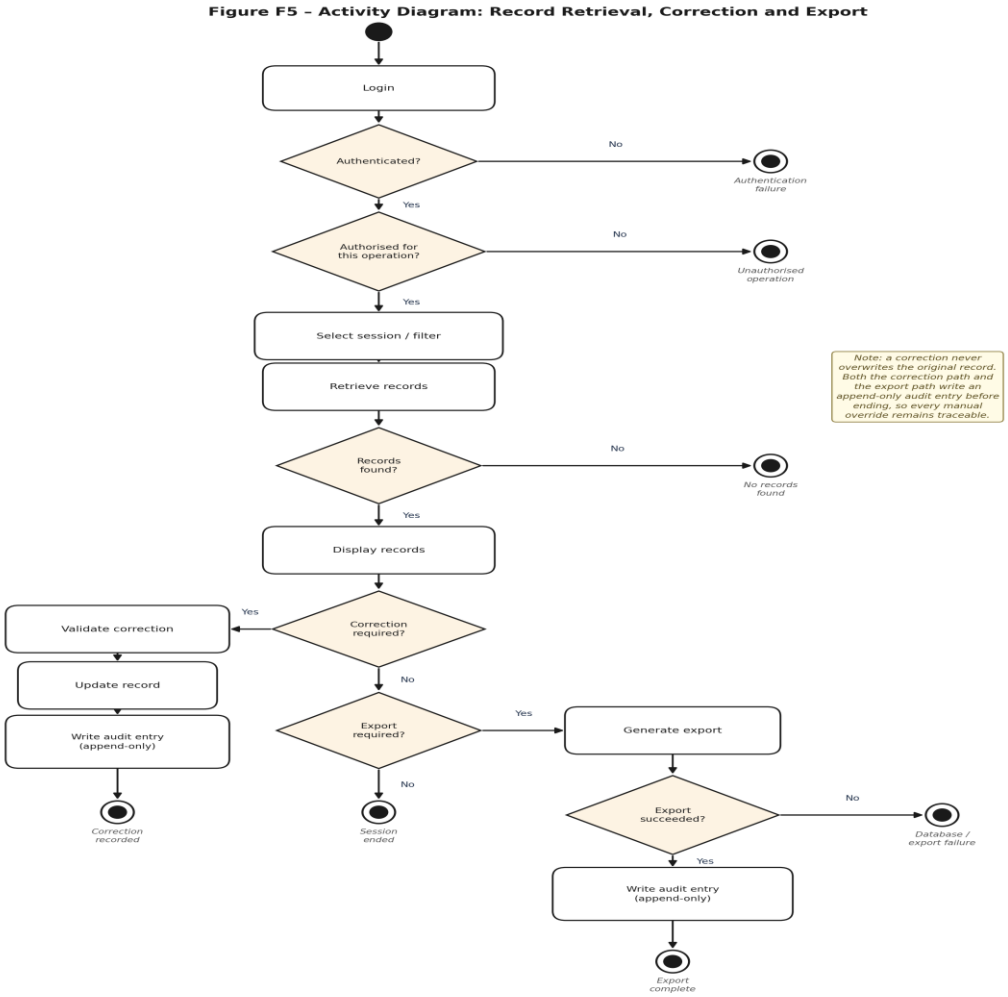
Status	Meaning	Triggered by	User feedback	Next status
QUEUED_OFFLINE	Attendance events have been stored locally because the server is unavailable.	Network failure after successful verification.	“Saved offline” / offline LED/confirmation beep	QUEUED_OFFLINE or SYNCED
SYNCED	Queued event has been successfully received and recorded by the server.	Successful synchronization.	“Synced” / success LED/confirmation beep	AWAITING_FINGER

SYNC_CONFLICT	Server cannot safely apply the queued event automatically.	Session conflict or other validation problem.	“Sync conflict” / warning LED/warning beep	FALLBACK_MANUAL
DUPLICATE_IGNORED	Queued event corresponds to attendance already recorded.	Duplicate/idempotency check.	“Already recorded” / warning LED / short beep	AWAITING_FINGER
FALLBACK_MANUAL	Lecturer resolves an event that could not be synchronized automatically.	Synchronization conflict.	“Lecturer confirmation” / warning LED / prompt beep	SYNCED or ERROR
ERROR	Offline storage or synchronization has failed.	Storage, hardware, or communication failure.	Error message/fault LED/error beep	QUEUED_OFFLINE or IDLE

F5 RECORD RETRIEVAL, CORRECTION AND EXPORT

This function is performed primarily through the administrator/lecturer dashboard rather than the terminal. It allows authorised users to view attendance records, correct genuine errors, and export attendance information. A correction must not simply erase the original history; the manual override must remain traceable through the append-only audit record.

Figure F5 – Activity Diagram: Record Retrieval, Correction and Export



F5 Process Flow Table

Process ID	Process name	Description	Actor	Precondition	Postcondition
F5-P1	**Authenticate user**	**The lecturer or administrator logs into the dashboard.**	**Lecturer/Administrator**	**Dashboard available.**	**User is authenticated.**
F5-P2	**Retrieve records**	**The user selects the required session or filters and retrieves attendance records.**	**Lecturer/Administrator/System**	**User is authorized.**	**Matching records are displayed.**
F5-P3	**Correct attendance record**	**An authorized user changes an incorrect attendance record.**	**Lecturer/Administrator**	**Record exists, and users have permission.**	**Corrected records are stored.**
F5-P4	**Record audit entry**	**The system records the correction or manual override in the audit table.**	**System**	**Correction completed.**	**Change is traceable.**
F5-P5	**Export attendance records**	**The user selects the required records and generates an export.**	**Lecturer/Administrator/System**	**User has export permission.**	**Attendance data is exported.**

F5 Status Flow Table

Status	Meaning	Triggered by	User feedback	Next status
IDLE	No active terminal attendance operation is taking place.	System ready.	Ready message / ready LED/none	SESSION_OPEN
VERIFIED	Attendance record has been accepted and is available as a valid record.	Successful attendance processing.	“Attendance recorded” / success LED/confirmation beep	AWAITING_FINGER

FALLBACK_MANUAL	A record has been confirmed or corrected through the permitted manual process.	Lecturer intervention.	Manual confirmation message / warning LED/confirmation beep	VERIFIED
ERROR	Requested retrieval, correction, or export cannot be completed.	Authentication, permission, database, or export failure.	Error message/fault LED/error beep	IDLE

Conclusion

The design of the Biometric Student Attendance and Verification System achieved the stated objective of developing a portable embedded attendance terminal integrated with networked biometric verification and attendance management. The proposed three-tier architecture separated the student-designed hardware terminal from the camera subsystem and server-side application, allowing fingerprint processing to remain at the terminal while computationally intensive face recognition was performed by the server. The design incorporated online and offline attendance operation, secure communication, duplicate prevention, auditability, and a non-biometric fallback, thereby addressing both operational continuity and the privacy requirements associated with biometric processing.

Three significant design trade-offs were made. First, face recognition was placed on the server rather than the ESP32-CAM, reducing the computational burden on the camera and terminal at the expense of increased dependence on network connectivity and server availability. Second, the system used 1 identification rather than 1:1 verification because students were not required to present a claimed identity, but the resulting increase in false-accept probability with population size justified limiting enrolment to approximately one class. Third, presentation attack detection (PAD) was deliberately bound to a supervised prototype evaluated using defined attack scenarios rather than attempting to provide certified liveness detection. This reduced development complexity while allowing PAD performance to be evaluated using APCER and BPCER in accordance with ISO/IEC 30107-3 [1].

Design remains subject to limitations. The prototype is limited to approximately 60 enrolled students, one attendance terminal, a local server environment, and supervised biometric operation. Face-recognition performance may also vary with lighting, image quality, and demographic factors [2], while the selected fingerprint and face technologies cannot be considered equivalent to certified access-control authentication. No measured acceptance results are claimed at the design stage; the stated performance values remain targets to be verified experimentally.

Future work may extend the system to multiple terminals, institutional information-system integration, broader access-control applications and certified liveness-detection technology. Such extensions would require additional scalability, security, privacy, interoperability and assurance considerations beyond the present project scope.

Glossary

Terms are listed alphabetically; bracketed numbers refer to the Bibliography.

1:1 Verification: A biometric comparison in which a claimed identity is confirmed by comparing one submitted sample against a single stored template, as used in the system's non-biometric fallback path.

1: N Identification: A biometric comparison in which a submitted sample is compared against every template enrolled for a class to determine identity without a prior identity claim; the system's primary matching topology.

APCER: Attack Presentation Classification Error Rate — the proportion of presentation attacks incorrectly classified as bona fide (genuine) presentations, as defined in ISO/IEC 30107-3 [2].

BMS: Battery Management System — protection circuitry that monitors and safeguards the terminal's rechargeable Li-ion battery pack against overcharge, overdischarge, and short-circuit conditions.

BPCER: Bona Fide Presentation Classification Error Rate — the proportion of genuine biometric presentations incorrectly classified as presentation attacks, as defined in ISO/IEC 30107-3 [2].
Device Registry: A server-side record of authorized attendance terminals and their associated shared secrets, allowing additional terminals to be added without redesigning the core attendance model.
Embedding: A fixed-length numerical vector produced by the dlib ResNet face-recognition model, representing the distinguishing features of a facial image for similarity comparison.
FAR: False Accept Rate — the probability that a biometric system incorrectly accepts an impostor as a match at a given decision threshold.
Failure-to-Enroll: The proportion of individuals for whom a usable fingerprint or facial biometric sample cannot be captured during enrolment.
FMR: False Match Rate — the probability that a submitted biometric sample is incorrectly matched to a template belonging to a different individual.
FNMR: False Non-Match Rate — the probability that a submitted biometric sample is incorrectly rejected as not matching its own enrolled template.
FRR: False Reject Rate — the probability that a genuine fingerprint sample is incorrectly rejected by the fingerprint subsystem.
Fusion Rule: The defined logic by which the primary fingerprint result and the secondary face-recognition result are combined to reach a single attendance decision.
HMAC: Hash-based Message Authentication Code — a cryptographic mechanism used to authenticate attendance-event payloads between the terminal and the server and to detect tampering or replay.
Idempotency Key: A client-generated identifier attached to an attendance event so that a retried or resynchronized submission produces the same recorded result rather than a duplicate attendance record.
JWT: JSON Web Token — a short-lived, digitally signed token used to authenticate and authorize lecturer and administrator sessions on the dashboard.
Nonce: A unique, single-use value included in each signed terminal request to prevent replay of a previously captured or queued message.
PAD: Presentation Attack Detection — methods used to determine whether a biometric sample originates from a genuine live subject rather than an artefact such as a printed photograph, screen replay or fabricated fingerprint.
POPIA: Protection of Personal Information Act 4 of 2013 [1] — South African legislation governing the lawful processing, retention and protection of personal information, including biometric data.
PSRAM: Pseudo-Static Random-Access Memory — additional external memory required by the ESP32-CAM to buffer image frames during capture and streaming.
RTC: Real-Time Clock — a battery-backed timekeeping component on the terminal used to timestamp attendance events captured while offline.
RTM: Requirements Traceability Matrix — a development artefact linking each stakeholder requirement to its architecture component, verification method and supporting evidence.
Special Personal Information: A category of personal information under POPIA that includes biometric data such as fingerprint and facial templates, and which is subject to stricter conditions for lawful processing.
Template: A mathematical representation of a fingerprint or facial feature set stored for matching purposes, as distinct from the original raw biometric image.
UART: Universal Asynchronous Receiver-Transmitter — the serial communication interface used between the ESP32 controller and the R307/AS608 fingerprint module.
Vertical Slice: A thin, end-to-end implementation of the system spanning one function from a single component through to a visible result, used as the first working increment of development.

Bibliography

References are listed in IEEE numeric style, in the order first cited in the text.
[1] Republic of South Africa, Protection of Personal Information Act 4 of 2013. Government Gazette, Cape Town, South Africa, Nov. 26, 2013. [Online]. Available: https://www.gov.za/documents/protection-personal-information-act.

[2] Information Regulator, Republic of South Africa, Regulations Relating to the Protection of Personal Information Act, 2018, GN 1383, Government Gazette 42110, Dec. 14, 2018. [Online]. Available: https://www.treasury.gov.za/POPIA/Final_Regulations_2018.pdf.

[3] ISO/IEC, ISO/IEC 30107-3:2023, Information Technology—Biometric Presentation Attack Detection—Part 3: Testing and Reporting, 2nd ed. Geneva, Switzerland: International Organization for Standardization, Jan. 2023. [Online]. Available: https://www.iso.org/standard/79520.html

[4] W. W. Royce, “Managing the development of large software systems: Concepts and techniques,” in Proc. IEEE WESCON, Los Angeles, CA, USA, Aug. 1970, pp. A/1–1–A/1–9.

[5] L. Rising and N. S. Janoff, “The Scrum software development process for small teams,” IEEE Softw., vol. 17, no. 4, pp. 26–32, Jul./Aug. 2000, doi: 10.1109/52.854065.

[6] K. Forsberg and H. Mooz, “The relationship of systems engineering to the project cycle,” in Proc. Joint Conf. National Council on Systems Engineering and American Society for Engineering Management, Chattanooga, TN, USA, Oct. 21–23, 1991, pp. 57–65.

[7] O. C. Z. Gotel and A. C. W. Finkelstein, “An analysis of the requirements traceability problem,” in Proc. 1st IEEE Int. Conf. Requirements Engineering, Colorado Springs, CO, USA, Apr. 1994, pp. 94–101, doi: 10.1109/ICRE.1994.292398.

[8] G. De Micheli and R. K. Gupta, “Hardware/software co-design,” Proc. IEEE, vol. 85, no. 3, pp. 349–365, Mar. 1997.

[9] D. E. King, “Dlib-ml: A machine learning toolkit,” J. Mach. Learn. Res., vol. 10, no. 60, pp. 1755–1758, 2009. [Online]. Available: https://www.jmlr.org/papers/v10/king09a.html.

[10] P. Grother, M. Ngan, and K. Hanaoka, Face Recognition Vendor Test (FRVT) Part 3: Demographic Effects, NIST Interagency or Internal Rep. 8280, National Institute of Standards and Technology, Gaithersburg, MD, USA, Dec. 2019, 81 pp., doi: 10.6028/NIST.IR.8280.

[11] C. Saraswat and A. Kumar, “An efficient automatic attendance system using fingerprint verification technique,” Int. J. Comput. Sci. Eng., vol. 2, no. 2, pp. 264–269, 2010.